실험 전 보고서

LAB6_MCU2

실험 주제 : SPI 통신 기반 MCU-FPGA 연동 및 데이터 송수신 구현

실험 대상 : STM32, FPGA, 오실로스코프

작성일 : 2026년 10월 25일

검증 환경 : STM32 개발보드, FPGA 개발보드, Serial Monitor(9600bps)

성명 : 권진혁

학과 : 전자전기컴퓨터공학부

학번 : 2023440007

1. 과제 8 - MCU 송신 파형 확인

· 실험 목적

본 실험의 목적은 SPI(Serial Peripheral Interface) 통신 프로토콜을 이용하여 STM32와 FPGA 간의 데이터 송수신 기능을 구현하고 검증하는 것이다. FPGA 내부에 SPI 송신기와 수신기를 설계하고 STM32와 연결하여 실제 데이터가 정상적으로 전달되는지를 확인한다. 또한 SCK, MOSI, MISO, CS 신호의 동작 원리를 이해하고 직렬 통신 인터페이스의 동작 과정을 학습하는 것을 목표로 한다.

· 핵심 코드 및 설명

```
void polling_routine(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();
    if (now_ms - last_ms < 1000U) return;
    last_ms = now_ms;

    static uint8_t tx = 0;
    tx++;

    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
                      SPI_CS_ece2_Pin, GPIO_PIN_RESET);
    HAL_StatusTypeDef result =
        uint8_t rx = 0;
        HAL_SPI_TransmitReceive(
            &hspi1,
            &tx,
            &rx,
            1,
            100
        );

    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
                      SPI_CS_ece2_Pin, GPIO_PIN_SET);

    if(rx == tx)
    {
        printf("PASS : RX=%02X\r\n", rx);
    }
    else
    {
        printf("FAIL : RX=%02X\r\n", rx);
    }
}
```

SPI Master 역할을 수행하는 STM32가 1초마다 데이터를 전송하고, 동시에 Slave로부터 데이터를 수신하여 통신 결과를 검증하는 함수이다. HAL_GetTick()을 이용하여 1초 주기로 동작하도록 설정하였으며, static uint8_t tx 변수를 사용하여 전송 데이터가 실행될 때마다 1씩 증가하도록 구현하였다. 이를 통해 고정된 값뿐만 아니라 다양한 데이터 패턴에 대한 SPI 통신의 정상 동작 여부를 확인할 수 있다.

데이터 전송 전에는 Chip Select(CS) 신호를 Low로 설정하여 SPI Slave를 활성화한다. 이후 HAL_SPI_TransmitReceive() 함수를 사용하여 1바이트 데이터를 송신하는 동시에 수신 데이터를 읽어 온다. SPI는 Full-Duplex 방식의 통신 규격이므로 송신과 수신이 동일한 클록 구간에서 동시에 수행된다. 통신이 종료되면 CS 신호를 High로 복귀시켜 해당 전송이 완료되었음을 알린다. 수신된 데이터는 전송한 데이터와 비교하여 통신의 정확성을 검사한다. 수신값(rx)과 송신값(tx)이 일치하면 UART 터미널에 "PASS" 메시지와 함께 수신 데이터를 출력하고, 일치하지 않을 경우 "FAIL" 메시지를 출력한다. 이를 통해 SPI 통신 과정에서 발생할 수 있는 데이터 손상이나 배선 오류를 쉽게 확인할 수 있으며, Master와 Slave 간 데이터 무결성을 검증할 수 있다.

· 실험 방법

MCU를 SPI 마스터 송신기로 설정한 후 제공된 송신 예제를 실행하여 1초마다 1바이트의 데이터를 전송하도록 구성한다. 이후 오실로스코프를 이용해 SCK, MOSI, CS 신호를 측정하고, 프로브의 GND를 MCU의 GND에 연결하여 기준 전압을 맞춘다. 먼저 송신 데이터를 0x55로 설정하여 SPI 통신 파형을 관찰하고, CS가 LOW인 동안 통신이 이루어지는지와 SCK가 8회의 상승 에지를 발생시키는지 확인한다. 이후 송신값을 0xA5, 0x00, 0xFF 등으로 변경하면서 MOSI 신호의 실제 비트 패턴이 전송 데이터와 일치하는지 비교한다. 또한 SCK의 주기

를 측정하여 SPI 클록 주파수를 계산하고 기록한다. 측정된 파형과 비트 순서를 분석하여 SPI Mode 0 방식이 정상적으로 동작하는지 검증한다.

· 예상 결과

SPI Master는 1초마다 1씩 증가하는 데이터를 Slave로 전송하게 된다. 첫 번째 전송에서는 0x01, 두 번째 전송에서는 0x02, 이후 0x03, 0x04와 같이 순차적으로 증가하는 데이터가 송신된다. 통신이 정상적으로 이루어질 경우 Slave는 수신한 데이터를 그대로 반환하거나 동일한 값을 송신하며, Master에서는 수신값(rx)과 송신값(tx)이 일치하여 UART 터미널에 다음과 같은 메시지가 출력될 것으로 예상된다. 반면, SPI 배선 오류, 클록 설정 불일치, Slave 동작 오류 등으로 인해 수신 데이터가 전송 데이터와 다를 경우에는 FAIL 메시지가 출력된다.

따라서 정상적인 실험 결과에서는 1초 간격으로 증가하는 데이터에 대해 연속적으로 PASS가 출력되며, 이를 통해 SPI 양방향 통신이 정상적으로 수행되고 데이터 무결성이 유지됨을 확인할 수 있을 것으로 예상된다. 또한 UART 시리얼 모니터에서 송수신 결과를 실시간으로 확인하여 SPI 인터페이스의 동작을 검증할 수 있다.

2. 과제 9 - MCU → MCU 수신 확인

· 실험 목적

SPI(Serial Peripheral Interface) 통신을 이용하여 두 개의 STM32 MCU 간 데이터 송수신을 수행하고, 송신 측에서 전송한 데이터가 수신 측에서 동일하게 수신되는지 확인하는 것이다. 또한 Master와 Slave 역할을 각각 수행해 봄으로써 SPI 통신의 동작 원리를 이해하고, MCU 간 직렬 통신의 신뢰성과 데이터 무결성을 검증하는 것을 목표로 한다.

· 핵심 코드 및 설명

```
void polling_routine(void)
{
    uint8_t rx = 0;
    HAL_StatusTypeDef result =
        HAL_SPI_Receive(&hspi1, &rx, 1U, 2000U);
    if (result == HAL_OK)
    {
        printf("RX=%02X\r\n", (unsigned int)rx);
    }
    else if (result == HAL_TIMEOUT)
    {
        printf("RX timeout: check sender, SCK, MOSI, CS and GND\r\n");
    }
    else
    {
        printf("RX error=%d\r\n", (int)result);
    }
}
```

SPI 통신을 통해 슬레이브 장치로부터 1바이트 데이터를 수신한 뒤, 수신 결과를 UART 터미널에 출력하는 폴링 방식의 함수이다. HAL_SPI_Receive() 함수를 이용하여 SPI1에서 데이터를 수신하고, 수신 성공 시(HAL_OK)에는 받은 데이터를 16진수 형식으로 출력한다. 반면, 지정된 시간(20초) 내에 데이터가 수신되지 않으면(HAL_TIMEOUT) SPI 클럭(SCK), MOSI, CS 연결 상태를 확인하라는 타임아웃 메시지를 출력한다. 또한 그 외의 오류가 발생한 경우에는 반환된 오류 코드를 함께 출력하여 문제를 진단할 수 있도록 하였다. 이를 통해 MCU가 SPI 슬레이브(또는 마스터)로부터 데이터를 정상적으로 수신하는지 확인하고, 통신 오류 발생 시 원인을 파악할 수 있다.

· 실험 방법

- 두 개의 STM32 보드를 준비하고 SPI 통신선(SCK, MOSI, MISO, NSS)과 GND

를 서로 연결한다.

- 한쪽 MCU를 SPI Master(송신 측), 다른 MCU를 SPI Slave(수신 측)로 설정한다.
- 송신 측 MCU에 과제 8에서 작성한 SPI 송신 프로그램을 다운로드한다.
- 수신 측 MCU에 SPI 수신 프로그램을 다운로드한다.
- 수신 측 MCU의 VCP UART를 PC와 연결하고 시리얼 모니터를 실행한다.
- Master MCU가 1초마다 증가하는 데이터를 전송하도록 실행한다.
- 수신 측 UART 창에 출력되는 값이 송신 측 데이터와 일치하는지 확인한다.
- Master와 Slave 역할을 서로 교체한 후 동일한 과정을 반복하여 양쪽 MCU 모두 송신 및 수신 기능을 검증한다.

· 예상 결과

SPI 통신이 정상적으로 동작할 경우 Master MCU에서 전송한 데이터가 Slave MCU에서 동일한 값으로 수신된다. 수신 측 VCP UART에는 송신 데이터와 같은 값이 순차적으로 출력되며, 데이터 손실이나 오류 없이 통신이 이루어짐을 확인할 수 있다. 또한 Master와 Slave의 역할을 교대하여 실험하더라도 동일한 결과가 나타나며, 두 MCU 모두 송신과 수신 기능이 정상적으로 동작함을 검증할 수 있을 것으로 예상된다.

3. 과제 10 - MCU → FPGA 수신 구현

· 실험 목적

SPI 통신을 이용하여 MCU가 전송한 데이터를 FPGA에서 정상적으로 수신하는 방법을 학습한다. FPGA에서 CS 신호 감지, SCK 상승 에지 검출, 시프트 레지스터를 이용한 8비트 데이터 수신 및 수신 완료 처리를 구현하고, 수신한 데이터를 UART로 출력하여 MCU의 송신 데이터와 일치하는지 확인함으로써 SPI 수신기의 동작을 검증한다.

· 핵심 코드 및 설명

Verilog

```
reg cs_d, sck_d;
reg [7:0] rx_data;
reg [2:0] bit_cnt;
reg rx_done;

always @(posedge clk) begin
    // 입력 동기화
    cs_d  <= cs;
    sck_d <= sck;

    rx_done <= 1'b0;

    // CS가 LOW인 동안 SPI 동작
    if(!cs) begin
        // SCK 상승 에지 검출
        if(!sck_d && sck) begin
            rx_data <= {rx_data[6:0], mosi};

            if(bit_cnt == 3'd7) begin
                bit_cnt <= 3'd0;
                rx_done <= 1'b1;      // 수신 완료
            end
            else begin
                bit_cnt <= bit_cnt + 1'b1;
```

CS 신호가 LOW가 되면 SPI 통신을 시작하며, SCK의 상승 에지를 검출할 때마다 MOSI의 데이터를 시프트 레지스터(rx_data)에 저장한다. 비트 카운터(bit_cnt)가 수신된 비트 수를 카운트하고, 8비트가 모두 수신되면 rx_done 신호를 1클록 동안 발생시켜 수신 완료를 알린다. 이후 rx_data는 UART 송신 모듈로 전달되어 PC 터미널에 출력되며, MCU가 전송한 데이터와 FPGA가 수신한 데이터가 일치하는지 확인할 수 있다. 이는 과제 10의 핵심인 CS 감지, SCK 상승 에지 검출, 8비트 수신, 수신 완료 처리를 하나의 블록으로 구현한 코드이다.

· 실험 방법

- STM32를 SPI Master 송신기로 설정
- FPGA를 SPI Slave 수신기로 설정
- FPGA 내부에 CS 감지, SCK 상승 에지 검출, 시프트 레지스터, 비트 카운터 및

UART 출력 회로를 구현
- TB에서 SPI 수신 모듈의 동작을 먼저 검증
- MCU에서 일정 주기로 데이터(예: 0x55, 0xA5 등)를 송신
- FPGA가 MOSI 신호를 통해 데이터를 수신
- 수신 완료 시 FPGA UART를 통해 PC 터미널에 데이터를 출력
- 터미널에 출력된 값과 MCU 송신값이 일치하는지 확인
- 다양한 데이터를 송신하면서 정상 수신 여부를 검증

· 예상 결과

MCU가 SPI를 통해 전송한 데이터가 FPGA에서 정상적으로 수신되며, 수신된 8비트 데이터가 UART를 통해 PC 터미널에 출력된다. MCU가 0x55, 0xA5, 0xFF, 0x00 등의 값을 전송할 경우 FPGA 터미널에서도 동일한 값이 출력되어 송수신 데이터의 일치가 확인된다. 또한 CS 신호가 활성화된 동안에만 데이터 수신이 수행되고, SCK의 상승 에지마다 정확하게 비트가 저장되어 8비트 수신 후 rx_done 신호가 발생함을 확인할 수 있다. 이를 통해 FPGA 기반 SPI Slave 수신기가 정상적으로 동작함을 검증할 수 있다.

4. 과제 11 - FGPA → MCU 송신 구현

· 실험 목적

FPGA를 SPI 마스터로 구성하여 SCK, CS, MOSI 신호를 직접 생성하고 MCU를 SPI 슬레이브로 설정하여 데이터를 수신하는 방법을 학습한다. 또한 SPI Mode 0 방식의 타이밍 조건을 만족하도록 송신 회로를 설계하고, FPGA가 전송한 데이터가 MCU의 UART 출력에 정상적으로 표시되는지를 확인하여 FPGA 기반 SPI 송신기의 동작을 검증한다.

· 핵심 코드 및 설명

Verilog

```
reg [7:0] tx_data = 8'h55;
reg [7:0] shift_reg;
reg [2:0] bit_cnt;
reg sck = 0;
reg cs = 1;
reg mosi = 0;

always @(posedge clk) begin
    if(send_start) begin
        cs <= 0;
        shift_reg <= tx_data;
        bit_cnt <= 3'd7;
        mosi <= tx_data[7];      // 첫 상승 에지 전에 MSB 준비
    end

    if(spi_clk_falling && !cs) begin
        shift_reg <= {shift_reg[6:0],1'b0};

        if(bit_cnt == 0) begin
            cs <= 1;             // 전송 종료
        end
        else begin
            bit_cnt <= bit_cnt - 1'b1;
            mosi <= shift_reg[6]; // 하강 에지에서 다음 비트 준비

        end
    end
end
```

FPGA가 SPI 마스터 역할을 수행하며 1 MHz 이하의 SPI 클록(SCK)을 생성한다. 전송 시작 시 CS를 LOW로 내려 슬레이브를 선택하고, 최상위 비트(MSB)를 MOSI에 먼저 출력한다. 이후 SCK의 하강 에지마다 시프트 레지스터를 이동시키면서 다음 비트를 MOSI에 출력한다. 총 8비트 전송이 완료되면 CS를 HIGH로 복귀시켜 통신을 종료한다. 이를 통해 SPI Mode 0 규격에 맞게 MCU가 상승 에지에서 데이터를 안정적으로 읽을 수 있도록 한다.

· 실험 방법

- FPGA에 SPI 마스터 송신 모듈을 구현
- SCK 주파수가 1 MHz 이하가 되도록 분주기를 설계

- 1초마다 1바이트(예: 0x55)를 송신하도록 설정
- 오실로스코프로 SCK, MOSI, CS 파형을 측정
- CS가 LOW인 동안에만 통신이 수행되는지 확인
- MOSI 비트 패턴이 송신 데이터와 일치하는지 확인
- FPGA의 SCK, MOSI, CS 및 GND를 MCU의 SPI 슬레이브 입력에 연결
- MCU는 슬레이브 수신 예제를 실행하여 데이터를 수신
- MCU의 VCP UART 터미널에 출력되는 값과 FPGA 송신값을 비교하여 정상 수신 여부를 확인

· 예상 결과

FPGA가 1초마다 SPI 데이터를 전송하면 SCK는 LOW 상태에서 대기하다가 통신 시에만 동작하고, CS는 LOW로 활성화된 후 8비트 전송이 완료되면 다시 HIGH로 복귀한다. MOSI에서는 송신 데이터에 해당하는 비트 패턴이 순차적으로 출력된다. MCU는 SPI 슬레이브로서 FPGA가 전송한 데이터를 정상적으로 수신하며 UART 터미널에 동일한 값을 출력한다. 예를 들어 FPGA가 0x55를 전송하면 MCU 터미널에는 RX=55, FPGA가 0xA5를 전송하면 RX=A5가 출력된다. 이를 통해 FPGA 기반 SPI 마스터 송신 회로와 MCU 수신 기능이 정상적으로 동작함을 확인할 수 있다.

5. 과제 12 - FPGA → FPGA 송수신 확인

· 실험 목적

FPGA 두 대를 SPI 통신으로 연결하여 한 FPGA는 마스터 송신기, 다른 FPGA는 슬레이브 수신기로 동작하도록 구현한다. 과제 11에서 설계한 SPI 송신 회로와 과제 10에서 설계한 SPI 수신 회로를 결합하여 데이터가 정상적으로 전송 및 수신되는지 확인하고, 수신 FPGA의 UART 출력을 통해 송수신 데이터의 일치 여부를 검증한다. 또한 역할을 교대하여 양쪽 FPGA 모두 송신기와 수신기의 동작을 확인한다.

· 핵심 코드 및 설명

Verilog

```
// 송신부 (Master)
always @(posedge clk) begin
    if(send_start) begin
        cs <= 1'b0;
        shift_tx <= tx_data;
        bit_cnt_tx <= 3'd7;
        mosi <= tx_data[7];
    end

    if(sck_falling && !cs) begin
        shift_tx <= {shift_tx[6:0], 1'b0};

        if(bit_cnt_tx == 0)
            cs <= 1'b1;
        else begin
            bit_cnt_tx <= bit_cnt_tx - 1'b1;
            mosi <= shift_tx[6];
        end
    end
end

// 수신부 (Slave)
always @(posedge clk) begin
    rx done <= 1'b0:
```

```
25
26      if(!cs && sck_rising) begin
27          rx_data <= {rx_data[6:0], mosi};
28
29          if(bit_cnt_rx == 3'd7) begin
30              bit_cnt_rx <= 3'd0;
31              rx_done <= 1'b1;
32          end
33          else begin
34              bit_cnt_rx <= bit_cnt_rx + 1'b1;
35          end
36      end
37  end
```

송신 FPGA는 SPI 마스터로 동작하여 SCK, CS, MOSI 신호를 생성하고 1초마다 1바이트의 데이터를 전송한다. 첫 번째 비트는 첫 SCK 상승 에지 이전에 MOSI에 준비되며 이후 하강 에지마다 다음 비트로 변경된다. 수신 FPGA는 SPI 슬레이브로 동작하여 CS가 LOW인 동안 SCK 상승 에지마다 MOSI 데이터를 읽어 시프트 레지스터에 저장한다. 8비트가 모두 수신되면 rx_done 신호를 발생시키고, 수신된 데이터를 UART 모듈로 전달하여 PC 터미널에 출력한다.

· **실험 방법**

- 송신 FPGA에 과제 11에서 구현한 SPI Master 송신 회로를 다운로드
- 수신 FPGA에 과제 10에서 구현한 SPI Slave 수신 회로를 다운로드
- 두 FPGA의 SCK, MOSI, CS, GND를 연결
- XDC 파일을 이용하여 각 FPGA의 입출력 핀을 설정
- 송신 FPGA가 1초마다 지정된 데이터(0x55, 0xA5 등)를 전송하도록 설정
- 오실로스코프로 SCK, MOSI, CS 파형을 확인하여 SPI Mode 0 규격이 지켜지는지 검증
- 수신 FPGA의 UART 터미널을 실행한다.
- UART에 출력되는 수신 데이터와 송신 FPGA의 전송값을 비교
- 역할을 교대하여 반대 FPGA가 송신기 역할을 수행하도록 하고 동일한 실험을 반복

· **예상 결과**

송신 FPGA가 SPI를 통해 전송한 데이터가 수신 FPGA에서 정상적으로 수신되며 UART 터미널에 동일한 값이 출력된다. 예를 들어 송신 FPGA가 0x55를 전송하면 수신 FPGA의 UART에는 RX=55가 출력되고, 0xA5, 0x00, 0xFF 등의 값을 전송하였을 때도 동일한 값이 표시된다. 또한 CS가 LOW인 동안에만 수신이 수행되며, SCK 상승 에지가 8회 발생한 후 수신 완료 신호가 생성된다. 역할 교대 후에도 동일한 결과가 나타나므로 FPGA 간 SPI 송수신 회로가 정상적으로 동작함을 확인할 수 있다.

6. 출처

[1] STMicroelectronics, STM32 Reference Manual (SPI Peripheral)

[2] Xilinx(AMD), Vivado Design Suite User Guide 및 FPGA Documentation

7. Git-Hub 및 커밋

https://github.com/UnnamedDays/ECE-II-Project

a1bd983